

Chương 5

LÝ THUYẾT MÃ

5.1. Đại cương về mã

5.1.1. Mã và mật mã

1) Ngôn ngữ và mã

- **Ngôn ngữ** trên **bảng chữ cái** Σ là tập con của vị nhóm tự do Σ^* ; mỗi phần tử của $x \in \Sigma^*$ là một xâu $a_1a_2\dots a_n$ gọi là **từ**, có độ dài là số $n \in \mathbb{N}$ chữ cái $a_i \in \Sigma$ của xâu đó, **phép nhân** xâu $x = a_1a_2\dots a_n$ với xâu $y = b_1b_2\dots b_m$ là ghép y vào bên phải x , $xy := a_1a_2\dots a_nb_1b_2\dots b_m$.

- **Mã** là một ngôn ngữ, mà với hai từ $x = a_1a_2\dots a_n$, $y = b_1b_2\dots b_m$ bất kỳ, $x = y$ khi và chỉ khi $m = n$ và $a_i = b_i$ ($i = 1, 2, \dots, n$).

- Mã gồm hai ánh xạ nghịch đảo nhau (D để **giải mã** và E để **mã hóa**), mã dùng trong thực tế chia ra:

*) **Mã máy**: Người dùng đưa bản tin vào máy bằng ngôn ngữ thông thường (nhờ bàn phím, tiếng nói, hình ảnh, ...); máy sẽ mã hóa thành tín hiệu *input*; tín hiệu đó được máy xử lý; các kết quả được máy tự giải mã rồi cho *output* ở dạng ngôn ngữ thông thường. Các ngôn ngữ lập trình cũng thuộc loại này.

*) **Mật mã**: Hai hàm D, E chỉ người gửi tin và người nhận tin biết. Một tập tin M được mã hóa thành $N = E(M)$, truyền N đi (tức là được ánh xạ tập tin M thành mã trên bảng chữ cái Σ nào đó) và gửi đi; người nhận dùng hàm D tạo nên $D(N) = D(E(M)) = M$ để nhận được bản tin gốc.

2) Bối cảnh về số học áp dụng cho lý thuyết mã.

- **Thặng dư bậc n** . Gọi số nguyên a là **thặng dư bậc n theo modulo m** (n nguyên dương), nếu tìm được số nguyên x mà $x^n \equiv a \pmod{m}$.

- **Khai căn bậc 2**. *) Cho p nguyên tố lẻ, a là số nguyên dương nhỏ hơn p . Nếu a là số **thặng dư bậc 2 theo mod p** , thì phương trình $x^2 \equiv a \pmod{p}$ có đúng 2 nghiệm là b và $p - b$. Thật vậy, nếu $b^2 \equiv a \pmod{p}$ thì $(p - b)^2 = p^2 - 2pb + b^2 \equiv b^2 \equiv a \pmod{p}$, chứng tỏ b và $p - b$ là nghiệm; nếu $b = p - b$, thì $b = p/2$, điều đó mâu thuẫn với p lẻ, nên b và $p - b$ là hai nghiệm khác nhau; nếu phương trình $x^2 \equiv a \pmod{p}$ có 3 nghiệm trở lên thì tìm được các nghiệm là x và $\pm y$ ($y \neq x$), khi đó $(x - y)(x + y) = x^2 - y^2 \equiv 0 \pmod{p}$, nhưng cả $x - y$ và $x + y$ nguyên tố cùng nhau với p , mâu thuẫn.

*) Nếu **p nguyên tố và $p \equiv 3 \pmod{4}$** , thì **$\pm a^{(p+1)/4} \pmod{p}$ là nghiệm của $x^2 \equiv a \pmod{p}$** . Thật vậy, $(\pm a^{(p+1)/4})^2 = a^{(p+1)/2} \equiv x^{2(p+1)/2} \equiv x^{p+1} \equiv x^p x \equiv ax \pmod{p}$ (Chú ý $x^p \equiv x$ là theo định lý nhỏ Fermat).

*) Bây giờ ta tìm nghiệm của phương trình $x^2 \equiv a \pmod{n}$, $n = pq$, p và q là các số nguyên tố đồng dư với 3 mod 4. Ta có $x^2 \equiv a \pmod{n}$ suy ra $x^2 = a + kn = a + kpq$, vậy $x^2 \equiv a \pmod{p}$ và $x^2 \equiv a \pmod{q}$. Giải hai phương trình $x^2 \equiv a \pmod{p}$ và $x^2 \equiv a \pmod{q}$ được $x \equiv \pm b \pmod{p}$ và $x \equiv \pm c \pmod{q}$. Cuối cùng nhờ định lý Trung Hoa về phần dư giải 4 hệ
$$\begin{cases} x \equiv b \pmod{p} \\ x \equiv c \pmod{q} \end{cases}, \begin{cases} x \equiv -b \pmod{p} \\ x \equiv c \pmod{q} \end{cases}, \begin{cases} x \equiv b \pmod{p} \\ x \equiv -c \pmod{q} \end{cases}, \begin{cases} x \equiv -b \pmod{p} \\ x \equiv -c \pmod{q} \end{cases}.$$

Thí dụ. Giải $x^2 \equiv 510 \pmod{713}$: Ta có $713 = 31 \cdot 23$; suy ra $x^2 \equiv 510 \equiv 14 \pmod{31}$ và $x^2 \equiv 510 \equiv 4 \pmod{23}$; do đó $x \equiv \pm 14^{(31+1)/4} \equiv \pm 14^8 \equiv \pm 18 \pmod{31}$ và $x \equiv \pm 4^{(23+1)/4} \equiv \pm 4^6 \equiv \pm 2 \pmod{23}$; $31^{-1} \pmod{23} \equiv 3 \pmod{23}$ và $23^{-1} \pmod{31} \equiv 27 \pmod{31}$. Áp dụng định lý Trung Hoa về phần dư và giải trong *Math.*: **Mod**[{ {18, 2}, {-18, 2}, {-18, -2}, {18, -2} } . {23*27, 31*3}, 31*23], được kết quả $x \equiv \{669, 416, 44, 297\} \pmod{713}$.

3) Giới thiệu một số mã và mật mã.

i) **Mã máy**. Để mã hoá 10 chữ số thập phân 0, 1, 2, ..., 9 thường dùng các **mã nhị phân** -

BCD (*Binary Code Deciben*), hoặc **mã Gray**, đó là các xâu 4 bit $vxyz$ cho ở cột 2, 3 bảng dưới. Người ta cũng dùng **mã nhị phân thừa 3**, **mã Gray thừa 3** đó là các xâu 4 bit $vxyz$ mà với $v = 0$ để mã cho 5 chữ số 0, 1, 2, 3, 4; với $v = 1$ để mã cho 5 chữ số còn lại: 5, 6, 7, 8, 9 cho ở cột 4 và 5 bảng dưới (như vậy mỗi nửa sẽ thừa ra 3 xâu bit). Trong bảng dưới, ở cột 2 mỗi chữ số thập phân k được mã bởi số nhị phân k tương ứng; cột 4 và 6 thì đối ngẫu qua trung tâm tức là đổi 1 thành 0, 0 thành 1; cột 5 thì trừ bit đầu ở nửa trên bằng 0, nửa dưới bằng 1 còn lại đối xứng qua trung tâm).

Chữ số	Cột	Mã nhị phân	Mã Gray	Mã nhị phân thừa 3	Mã Gray thừa 3	Mã (2,4,2,1)
		2	3	4	5	6
0		0000	0000	0011	0010	0000
1		0001	0001	0100	0110	0001
2		0010	0011	0101	0111	0010
3		0011	0010	0110	0101	0011
4		0100	0110	0111	0100	0100
5		0101	0111	1000	1100	1011
6		0110	0101	1001	1101	1100
7		0111	0100	1010	1111	1101
8		1000	1100	1011	1110	1110
9		1001	1101	1100	1010	1111

Mã nhị phân thuộc loại mã 4 bit có trọng số, trọng số của mã này là (8, 4, 2, 1) tức là chữ số k được biểu diễn thành xâu $vxyz$ thì $k = 8v + 4x + 2y + 1z$. Cột cuối của bảng trên cũng là một mã có trọng số, trọng số ở đây là (2, 4, 2, 1) $k = 2v + 4x + 2y + 1z$, tuy nhiên có chữ số sẽ được biểu diễn 2 dạng mã, khi đó người làm mã quyết định lấy một dạng để sử dụng (các số 5, 6, 7 có thể mã theo trọng số (2, 4, 2, 1) dưới dạng khác mà ở đây không được sử dụng đó là 0101, 0110, 0111). Mã Gray và mã Gray thừa 3 có tính chất đặc biệt là hai chữ số liên tiếp được mã bởi hai xâu sai khác nhau đúng 1 bit, điều đó có lợi cho việc chống nhiễu khi truyền tin. Trong bảng mã trên, mỗi cột thừa ra 6 xâu 4 bit, đó là các xâu không cần quan tâm. Một cách biểu diễn các bảng mã ở một dạng khác (bìa Karnaugh) để nhớ hơn và thuận tiện cho người dùng.

ii) **Mã ASCII** (*American National Code Information Interchange*) là một loại mã máy được dùng phổ biến trên thế giới và được dùng mã hoá bàn phím thông dụng, nhằm đưa dữ liệu vào máy. Vì tính phổ dụng của mã ASCII nên các bộ mã khác cho dù được thiết kế tốt hơn vẫn không được truyền bá. Mã ASCII được dùng để mã các chữ cái, các dấu và các chữ số. Mã ASCII dùng các xâu 8 bit $x_0x_1x_2 \dots x_7$, với bit đầu $x_0 = 0$ cho bảng mã dưới đây, với $x_0 = 1$ để dùng mã hoá các ký tự đồ hoạ. Nhờ mã ASCII dễ dàng điều khiển máy tính thông qua các ngôn ngữ lập trình như Pascal, C, ... mà không phải dùng đến mã máy. Tuy nhiên, máy chỉ hoạt động theo ngôn ngữ máy (được mã hoá bằng mã nhị phân).

Ý nghĩa của các ký hiệu chữ trong bảng mã ASCII:

NUL	Số 0, không, vô hiệu	BS	Lùi 1 khoảng ký tự
SOH	Bắt đầu của tiêu đề	HT	Kẻ bảng hướng ngang
STX	Bắt đầu của hành văn	LF	Chuyển động
ETX	Kết thúc của hành văn	VT	Kẻ bảng hướng dọc
EOT	Kết thúc truyền tin	FF	Điều khiển chạy giấy
ENQ	Hỏi	CR	Quay về dòng đầu
ACK	Thừa nhận	SO	Dịch ra (Shift out)
BEL	Chuông	SI	Dịch vào (Shift in)
DLE	Chuyển mã	EM	Hết giấy
DC1	Điều khiển thiết bị 1	SUB	Trừ
DC2	Điều khiển thiết bị 2	ESC	Chuyển mã
DC3	Điều khiển thiết bị 3	FS	Dấu phân cách file

DC4	Điều khiển thiết bị 4	GS	Dấu phân cách gói
NAK	Phủ định	RS	Dấu phân cách ghi
SYN	Đồng bộ	US	Dấu phân cách đơn vị
ETB	Kết thúc truyền gói tin	SP	Khoảng trống ký tự
CAN	Hủy bỏ	DEL	Hủy bỏ lùi

$x_1x_2x_3$ $x_4x_5x_6x_7$	000	001	010	011	100	101	110	111
0000	NUL	DEL	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	”	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

- Mã hóa bằng FFT.

Thuật toán **biến đổi Fourier nhanh** (*Fast Fourier-Transform*) giúp mã hóa thông tin để truyền đi xa mà có thể giải mã tức thì.

Để làm điều đó người ta: *i)* Chia tập tin thành những xâu (x_j) độ dài $T = 2^m$ ($m \in \mathbb{N}$) dùng mã hóa thông thường (chẳng hạn mã ASCII) biến thành xâu T số thực hoặc phức;

ii) Dùng FFT biến thành xâu T số mới, truyền đi hai bản tin là bản gốc và bản FFT, người nhận tin sẽ có hai bản để so sánh, khắc phục lỗi khi truyền.

Cho xâu gồm T số thực hoặc phức $x = (x_j), j = 0, \dots, T-1$ ($T \in \mathbb{N}^+$). **Phép biến đổi Fourier rời rạc** của xâu x đã cho là xâu $X = (X_k), k = 0, \dots, T-1$ mà $X_k := \sum_{j=0}^{T-1} x_j W^{kj}$ (với $k \in \mathbb{Z}, i = \sqrt{-1}$ là đơn vị ảo, $W := e^{-i2\pi/T}$). Chú ý là $\forall m \in \mathbb{Z}$, ta có $W^{mT} = (W^T)^m = 1$; $W^{j+mT} = W^j$; do đó $X_{k+mT} = X_k$, tức là X_k tuần hoàn

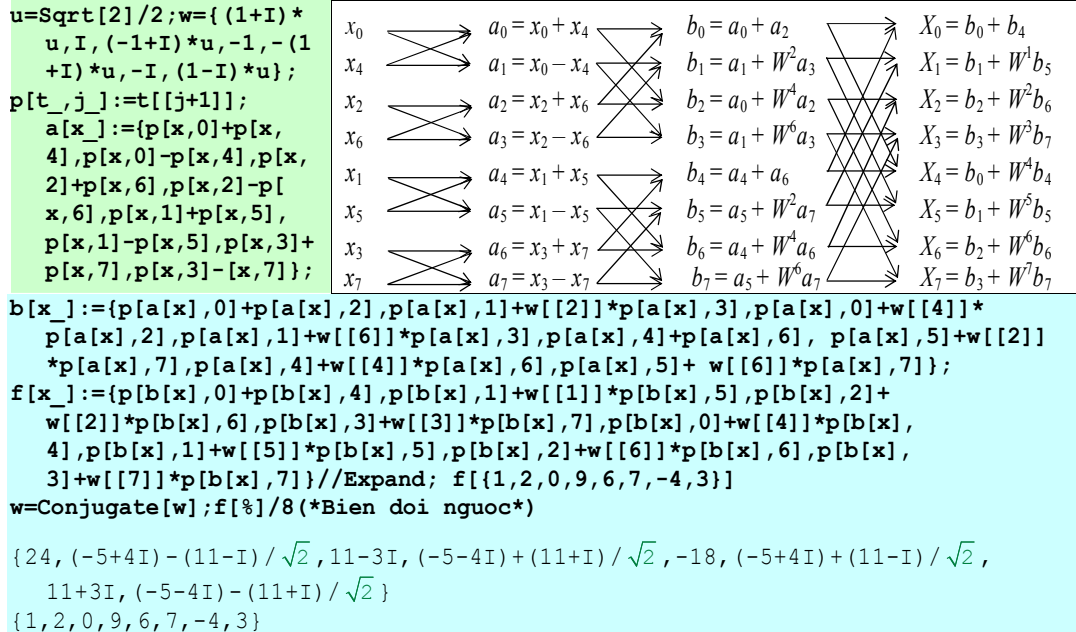
theo biến k với chu kỳ T . Công thức **biến đổi Fourier rời rạc ngược** là $x_j = \sum_{k=0}^{T-1} \frac{X_k}{T} W^{-jk}, j = 0, \dots, T-1$

cũng là biến đổi Fourier rời rạc.

Thuật toán biến đổi Fourier nhanh của Cooley và Tukey (1965): Giả sử $T = 2^m$ ($m \in \mathbb{N}$), đặt $S = T/2$. Khi đó $W^2 = e^{-i2\pi/T \cdot 2} = e^{-i2\pi/S}$, nên $X_k = \sum_{j=0}^{T-1} x_j W^{kj} = \sum_{h=0}^{S-1} x_{2h} W^{2hk} + W^k \sum_{h=0}^{S-1} x_{2h+1} W^{2hk}$; tức là

$X_k = X_{0k} + W^k X_{1k}$, trong đó X_{0k} và X_{1k} là biến đổi Fourier của xâu S số $(x_0, x_2, \dots, x_{T-2})$ và $(x_1, x_3, \dots, x_{T-1})$. Tiếp tục ta lại có $X_{0k} = X_{00k} + W^{2k} X_{01k}, X_{1k} = X_{10k} + W^{2k} X_{11k}; X_{00k} = X_{000k} + W^{4k} X_{001k}, X_{01k} = X_{010k} + W^{4k} X_{011k}, X_{10k} = X_{100k} + W^{4k} X_{101k}, X_{11k} = X_{110k} + W^{4k} X_{111k}; \dots$ Vì lũy thừa nguyên của W chỉ

có đúng T giá trị khác nhau nên FFT rất có hiệu quả khi $T = 2^m$. Với $m = 3$ ta có sơ đồ Thuật toán FFT (có 13 phép nhân và 6 phép lũy thừa) ở bảng sau.



- Mã Caesar.

Nhà quân sự thiên tài thời cổ đại đã dùng *mật mã* như sau: Ứng các chữ cái $a, b, \dots, z$ lần lượt theo thứ tự: 00, 01, ..., 25; rồi cộng thêm 3 thành 03, 04, ..., 28; sau đó lấy thẳng dư theo mod26, tức là $a, b, c, d, \dots, u, v, w, x, y, z$ có mã hóa là 03, 04, 05, 06, ..., 23, 24, 25, 00, 01, 02 và mã hóa bản tin rồi truyền đi. Người nhận đổi các số 00, 01, 02, 03, ..., 23, 24, 25 thành $x, y, z, a, \dots, u, v, w$ và có được văn bản gốc. Mã Caesar thực chất là dùng hàm $x \mapsto x+3 \pmod{26}$ để mã hóa và hàm $y \mapsto y-3 \pmod{26}$ để giải mã.

- Mã aphine.

Mã Caesar là mã dãy, tổng quát ta có mã aphine: $x \mapsto nx + a \pmod{m}$ với a tùy ý và $\gcd(n, m) = 1$, với $m = 26$ ta có $\phi(m) = m(1 - 1/2)(1 - 1/13) = 12$ giá trị n : 1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25. Mã này là loại *mật mã* tương đối đơn giản nên dễ bị lộ (có thể dựa vào tần suất để nhận ra cách mã hóa từ đó giải được mã).

- Mã Vigenere.

Mã bộ chữ cái thành 00, 01, ..., 25 được bản tin số; lấy ra k số $b_1, b_2, \dots, b_k$ cố định thuộc tập $\{00, 01, \dots, 25\}$ (thường là thứ tự các chữ cái của một từ nào đó trong bảng alphabet), bản tin số được chia thành các đoạn liên tiếp độ dài $2k$, tức là đoạn thứ n gồm k chữ cái, thể hiện bằng k cặp số: $a_{n1}, a_{n2}, \dots, a_{nk}$; **cộng** a_{ni} với b_i và lấy đồng dư theo mod26. Khi giải mã ta cần lần lượt **trừ** mỗi chữ cái cho b_i , rồi lấy mod26.

Nếu bản gốc M là một xâu bit, thì có thể lấy ngẫu nhiên một xâu bit A độ dài k làm khóa, chia M thành các đoạn độ dài k , rồi mã hóa bằng cách cộng mỗi đoạn đó với A , được bản mã hóa là N , rồi truyền đi, khi giải mã chỉ cần chia N thành các đoạn độ dài k , rồi cộng mỗi đoạn đó với A . **Thí dụ.** $M = 1101000\ 1010001\ 0010101$; $A = 1011001$, thì $N = 0110001\ 0001000\ 1001100$.

- Mã khối (Mã Hill).

Chống dùng tần suất để thám mã người ta dùng mã khối. Ứng mỗi chữ cái $a, b, \dots, z$ với số nguyên 00, 01, ..., 25 (ở đây $m = 26$); bản tin có n chữ cái viết liền thành $2n$ chữ số; chia bản tin thành các đoạn có k cặp số liên tiếp (ứng với k chữ cái) tạo ra vector cột x (k thành

phần, mỗi thành phần có 2 chữ số, ứng với 1 chữ cái); dùng matrix vuông A cấp k , nhân $A.x \pmod{m}$ được vector y rồi gửi đi. Dùng $A^{-1}.y \pmod{m}$ để giải mã. Điều đó có nghĩa là giải hệ phương trình $Ax = y \pmod{m}$, hệ phương trình này có nghiệm khi và chỉ khi $\Delta = \det A$ nguyên tố cùng nhau với m .

- Mã hoán vị (*Rain Fence*).

Lấy một số nguyên dương k , xếp các ký tự của bản gốc thành các hàng từ trên xuống của một matrix k cột (số hàng tùy thuộc độ dài bản tin gốc, hàng cuối nếu không đủ k ký tự thì bổ sung bằng chữ “z”), thực hiện hoán vị các cột, rồi xếp các ký tự của các cột liên tiếp nhau sẽ được bản tin đã mã hóa.

- Mã ngẫu nhiên.

Giả sử bản tin đã mã hóa có dạng một xâu bit M . Chia M thành m đoạn nhỏ một cách tùy ý, lấy m số nguyên dương $k_i (\leq 100)$. Tạo xâu bit N bằng cách: Trước mỗi bit của đoạn M_i (đoạn thứ i của M) chèn thêm k_i bit tùy ý tạo thành xâu N_i , N là dãy vector (k_i, N_i) , truyền N đi. Giải mã bằng cách chia N_i thành các xâu độ dài k_i+1 , xóa k_i bit đầu của , giữ lại bit thứ k_i+1 của đã thêm vào trước mỗi xâu ấy chỉ giữ lại bit thứ k_i+1 .

Xếp thứ tự 26 chữ cái																									
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
Mã Caesar. $x + 3 \pmod{26}$																									
03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	00	01	02
Mã affine. $7x + 5 \pmod{26}$																									
5	12	19	00	07	14	21	02	09	16	23	04	11	18	25	06	13	20	01	08	15	22	03	10	17	24

Mã Vigenère. Từ khóa là “*hoadep*” ứng với 6 số b_i : {07, 14, 00, 03, 04, 15}. Văn bản gốc “*paraboloidhy-perbolic*” có 20 chữ, được chia thành 3 đoạn 6 chữ và 2 chữ cuối: *parabo*, *loidhy*, *perbol*, *ic*. Mã hóa thành {15+07, 00+14, 17+00, 00+03, 01+04, 14+15}, {11+07, 14+14, 08+00, 03+03, 07+04, 24+15}, {15+07, 04+14, 17+00, 01+03, 14+04, 11+15}, {08+07, 11+14}; lấy mod26 được xâu: 221417030504 180308061113 221817041800 1525.

Mã Hill. Lấy matrix $A = \begin{bmatrix} 1 & 4 \\ 2 & 9 \end{bmatrix}$ có $\det A = 1 \in \mathbb{Z}_{26}^*$ (tức là A khả nghịch); các ký tự của “*paraboloid*” xếp 2

chữ một, tạo ra 5 vector cột của matrix $\begin{bmatrix} p & r & b & l & i \\ a & a & o & o & d \end{bmatrix} = \begin{bmatrix} 15 & 17 & 01 & 11 & 08 \\ 00 & 00 & 14 & 14 & 03 \end{bmatrix}$; nhân A với các vec-

tor đó được $\begin{bmatrix} 15 & 17 & 57 & 67 & 20 \\ 30 & 34 & 128 & 148 & 43 \end{bmatrix} = \begin{bmatrix} 15 & 17 & 05 & 15 & 20 \\ 04 & 08 & 24 & 18 & 17 \end{bmatrix} \pmod{26}$; xếp lại thành một xâu, ta

được: 15041708052415182017. Để giải mã ta tìm $A^{-1} = \begin{bmatrix} 9 & -4 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} 9 & 22 \\ 24 & 1 \end{bmatrix} \pmod{26}$.

- **Mã mũ (Mã Pohlig-Hellman).** Mã này được *Pohlig* và *Hellman* đưa ra năm 1978. Đầu tiên mã bộ chữ cái thành 00, 01, ..., 25. Lấy số p , e với p là số nguyên tố lẻ, e nguyên tố cùng nhau với $p-1$. Tìm d sao cho $de \equiv 1 \pmod{p-1}$. Chọn m thỏa mãn:

$$26(100^m + 100^{m-1} + \dots + 100^{m-m}) < p < 26(100^{m+1} + 100^m + 100^{m-1} + \dots + 100^{m-m})$$

Tách văn bản thành những xâu độ dài $2m$ (xâu cuối ngắn hơn $2m$ thì thêm các cặp số 25 cho đủ $2m$ ký tự). Mã hóa các xâu M bằng hàm $N = E(M) = M^e \pmod{p}$. Truyền đi. Người nhận sẽ giải mã bằng hàm $D(N) = N^d \pmod{p}$ tìm ra M .

Thí dụ. Chọn $p = 10007$; $e = 1201$ (vì $\gcd(p-1, e) = 1$); $d = \text{Position}[\text{Table}[\text{Mod}[1201n, p-1], \{n, 1, 10006\}], 1][[1, 1]] = 1433$. Chọn $m = 1$, vì $26 \cdot 101 = 2626 < p < 26 \cdot 10101 = 262626$. Văn bản gốc “*paraboloid*” số

hóa và chia thành xâu con độ dài $2m = 2$, được 10 xâu con: $A = \{15,00,17,00,01,14,11,14,08,03\}$; mã hóa được $B = \text{Mod}[A^e, p] = \{1215, 0, 3120, 0, 1, 1297, 2369, 1297, 9771, 351\}$. Giải mã: $\text{Mod}[B^d, p]$.

2.2.8. Bổ sung về mã

1) *Cơ sở lý thuyết mã*. Chúng ta sẽ bàn về Lý thuyết mã đại số, đó là lĩnh vực dựa trên các cấu trúc đại số, lý thuyết xác suất và lý thuyết tổ hợp. Chúng ta bắt đầu từ mô hình truyền tin kỹ thuật số, tức là truyền các xâu bit 0-1 và do nhiễu mà một tín hiệu nhận được không trung thành với tín hiệu truyền đi và mắc một lỗi s đó. Để khắc phục lỗi phạm phải khi truyền tin mà ở đây chúng ta dùng kênh nhị phân đối xứng, tức là xác suất truyền sai tín hiệu 0 và 1 đều là p , xác suất đúng là $1 - p$.

Thí dụ. Truyền đi xâu $c = 10110 \in \mathbb{Z}_2^5$ với $p = 5\%$, giả thiết việc truyền mỗi bit là sự kiện độc lập, do đó xác suất truyền đúng c là $0,95^5 = 0,77$. Nếu bên nhận nhận được $r = 00110$ (sai 1 bit) thì xác suất là $0,05 \times 0,95^4 = 0,041$; nếu $r = 00100$ (sai 2 bit) thì xác suất là $0,05 \times 0,95^2 \times 0,05 \times 0,95 = 0,002$ rất nhỏ. Xác suất xảy ra r có 2 bit sai là $C_5^2 \times 0,05^2 \times 0,95^3 = 0,021$.

Giả sử $c \in \mathbb{Z}_2^n$, xác suất truyền sai một bit là p , truyền c đi, nhận về r thì $r = c \oplus e$, với e là xâu mà bit 1 ứng với bit của c bị truyền sai. Xác suất một lần truyền có k lỗi là $p^k(1-p)^{n-k}$, xác suất xảy ra k lỗi là $C_n^k p^k(1-p)^{n-k}$.

Trong thí dụ trên xác suất truyền sai không quá một bit là $0,95^5 + C_5^1 \times 0,05 \times 0,95^4 = 0,977$, chứng tỏ rằng với p nhỏ thì xác suất xảy ra lỗi là khá bé. Trong thực tế khi $p < 10^{-5}$ thì coi hệ thống truyền tin là tốt, nhưng khi nghiên cứu ta chỉ giả thiết $p < 0,5$.

Một số mã ra đời trước mã Hamming

Mã khối.

Để tăng độ chính xác khi truyền tin người ta dùng **mã khối (m, n)** để truyền các tin là xâu bit c độ dài m ta nối thêm $n - m$ bit thành xâu C độ dài n ($> m$) bằng một đơn ánh $E: \mathbb{Z}_2^m \rightarrow \mathbb{Z}_2^n$, sau khi truyền tin và giải mã sẽ dùng ánh xạ $D: \mathbb{Z}_2^n \rightarrow \mathbb{Z}_2^m$ để tìm bản tin gốc.

Mã chẵn lẻ

Mã chẵn lẻ thêm một bit vào trong dữ liệu, và bit cho thêm này cho biết số lượng bit có giá trị 1 của đoạn dữ liệu nằm trước là một số chẵn hay một số lẻ. Nếu một bit bị thay đổi trong quá trình truyền dữ liệu, giá trị chẵn lẻ trong thông điệp sẽ thay đổi và do đó có thể phát hiện được lỗi (Chú ý rằng bit bị thay đổi có thể lại chính là bit kiểm tra). Theo quy ước chung, bit kiểm tra có giá trị bằng **1** nếu số lượng bit có giá trị một trong dữ liệu là một số **lẻ**, và giá trị của bit kiểm tra bằng **0** nếu số lượng bit có giá trị một trong dữ liệu là một số **chẵn**. Nói cách khác, nếu đoạn dữ liệu và bit kiểm tra được gộp lại cùng với nhau, số lượng bit có giá trị bằng 1 luôn luôn là một số chẵn.

Thí dụ, với mã khối $(9,8)$ ánh xạ E **ứng mỗi xâu $x_1x_2 \dots x_8$ với $x_1x_2 \dots x_8x_9$ bằng cách nối thêm $x_9 = x_1 \oplus x_2 \oplus \dots \oplus x_8$** (chẳng hạn $E(10010101) = 100101010$, $E(10010111) = 100101111$); khi đó truyền sai không quá 1 bit có xác suất là $(1-p)^9 + C_9^1 p(1-p)^8$ (với $p = 0,001$ thì xác suất đó là 0,99996417), chú ý rằng các xâu $x_1x_2 \dots x_8x_9$ đều có một số chẵn bit 1 – mã khối này được gọi là **mã kiểm tra chẵn lẻ**.

Việc kiểm tra dùng mã chẵn lẻ là một việc không được chắc chắn cho lắm, vì nếu số bit bị thay đổi là một số chẵn (2, 4, 6 - cả hai, bốn hoặc sáu bit đều bị hoán vị) thì mã này không phát hiện được lỗi. Hơn nữa, mã chẵn lẻ không biết được bit nào là bit bị lỗi, kể cả khi nó phát hiện là có lỗi xảy ra. Toàn bộ dữ liệu đã nhận được phải bỏ đi, và phải truyền lại từ đầu. Trên một kênh truyền bị nhiễu, việc truyền nhận thành công có thể mất rất nhiều thời gian, nhiều khi còn không truyền được nữa. Mặc dù việc kiểm tra bằng mã chẵn lẻ không được tốt cho lắm, song vì nó chỉ dùng 1 bit để kiểm tra cho nên nó có số tổng phí (*overhead*) thấp

nhất, đồng thời, nó cho phép phục hồi bit bị thất lạc nếu người ta biết được vị trí của bit bị thất lạc nằm ở đâu.

Mã hai-trong-năm

Trong những năm của thập niên kỷ 1940, Bell có sử dụng một mã hiệu phức tạp hơn một chút, gọi là mã hai-trong-năm (*two-out-of-five code*). Mã này đảm bảo mỗi một khối 5 bit (còn được gọi là *khối-5*) có chính xác hai bit có giá trị bằng 1. Máy tính có thể nhận ra là dữ liệu nhập vào có lỗi nếu trong một khối 5 bit không 2 bit có giá trị bằng 1. Tuy thế, mã hai-trong-năm cũng chỉ có thể phát hiện được một đơn vị bit mà thôi; nếu trong cùng một khối, một bit bị lộn ngược thành giá trị 1, và một bit khác bị lộn ngược thành giá trị 0, quy luật hai-trong-năm vẫn cho một giá trị đúng (*remained true*), và do đó nó không phát hiện là có lỗi xảy ra.

Mã chập 3 (Tái diễn dữ liệu)

Người ta còn dùng mã khối $(3m, m)$ – **mã chập ba**: mỗi **xâu x độ dài m lặp lại hai lần thành xâu xxx độ dài $3m$** và truyền đi, khi nhận được chỉ cần so sánh 3 đoạn xâu để khắc phục lỗi dựa vào đa số, tuy nhiên mã này tăng thời gian truyền tin lên quá nhiều (3 lần). Nếu bit dữ liệu cần được truyền có giá trị bằng 1, một *mã tái diễn* $n = 3$ sẽ cho truyền gửi giá trị "111". Nếu ba bit nhận được không giống nhau, thì hiện trạng này báo cho ta biết rằng, lỗi trong truyền thông đã xảy ra. Nếu kênh truyền không bị nhiễu, tương đối đảm bảo, thì với hầu hết các lần truyền, trong nhóm ba bit được gửi, chỉ có một bit là bị thay đổi. Do đó các nhóm 001, 010, và 100 đều tương đương cho một bit có giá trị 0, và các nhóm 110, 101, và 011 đều tương đương cho một bit có giá trị 1. Một mã có khả năng tái dựng lại thông điệp gốc trong một môi trường nhiễu lỗi được gọi là mã "sửa lỗi" (*error-correcting code*).

Tuy nhiên, những mã này không thể sửa tất cả các lỗi một cách đúng đắn hoàn toàn. Chẳng hạn chúng ta có một ví dụ sau: nếu một kênh truyền sai hai bit và do đó máy nhận thu được giá trị "001", hệ thống máy sẽ phát hiện là có lỗi xảy ra, song lại kết luận rằng bit dữ liệu gốc là bit có giá trị bằng 0. Đây là một kết luận sai lầm. Nếu chúng ta tăng số lần các bit được nhắc lại lên 4 lần, chúng ta có thể phát hiện tất cả các trường hợp khi 2 bit bị lỗi, song chúng ta không thể sửa chữa chúng được (số phiếu bầu "hòa"); với số lần nhắc lại là 5 lần, chúng ta có thể sửa chữa tất cả các trường hợp 2 bit bị lỗi, song không thể phát hiện ra các trường hợp 3 bit bị lỗi. Nói chung, mã tái diễn là một mã hết sức không hiệu quả, giảm công suất xuống 3 lần so với trường hợp đầu tiên trong ví dụ trên của chúng ta, và công suất làm việc giảm xuống một cách nhanh chóng nếu chúng ta tăng số lần các bit được nhắc lại với mục đích để sửa nhiều lỗi hơn.

2) Khoảng cách Hamming.

- **Hàm trọng lượng** $wt(x)$ được định nghĩa là số bit bằng 1 của xâu x (chẳng hạn $wt(100110) = 3$; $wt(010100) = 2$).

- **Khoảng cách Hamming** $d(x, y)$ giữa hai xâu x và y được định nghĩa là $d(x, y) := wt(x \oplus y)$, tức là số bit tương ứng của x và của y khác nhau.

Thí dụ. với $x = 0110001$; $y = 1101000$ ta có $wt(x) = 3$; $x \oplus y = 1011001$; $d(x, y) = 4$.

Định lý: Hàm $d(x, y)$ thỏa mãn 3 tiên đề khoảng cách.

*) $d(x, y)$ không âm và $d(x, y) = 0$ khi và chỉ khi $x = y$ (*tiên đề xác định dương*);

*) $wt(x \oplus y) = wt(y \oplus x)$, tức là $d(x, y) = d(y, x)$ (*tiên đề đối xứng*);

*) $wt(x \oplus y) = wt(x) + wt(y) - 2wt(x \otimes y) \leq wt(x) + wt(y)$, suy ra $wt(x \oplus z) = wt(x \oplus (y \oplus y) \oplus z) = wt((x \oplus y) \oplus (y \oplus z)) \leq wt(x \oplus y) + wt(y \oplus z)$, tức là $d(x, z) \leq d(x, y) + d(y, z)$ (*tiên đề tam giác*).

Gọi tập $S(x_0, r) := \{x \mid d(x, x_0) \leq r\}$ là **hình cầu** tâm tại x_0 bán kính r .

Thí dụ. Trong $\mathbb{Z}_2^4$, $x_0 = 1001$ thì $S(x_0, 1) = \{0001, 1000, 1101, 1011\}$, $S(x_0, 2) = S(x_0, 1) \cup \{0000, 0011, 0101, 1100, 1111, 1010\}$.

0000	0001	0011	0010
0100	0101	0111	0110
1100	1101	1111	1110
1000	1001	1011	1010

Có thể chứng minh hai khẳng định sau:

- Giả sử E là một hàm mã hóa, tập tin là $W \subset \mathbb{Z}_2^m$ và tập mã $E(W) = C \subset \mathbb{Z}_2^n$ ($m < n$). Ta có thể **phát hiện tất cả các lỗi** truyền tải có trọng lượng $\leq k$ khi và chỉ khi khoảng cách tối thiểu giữa các từ của tập mã không nhỏ hơn $k + 1$.

- Cho E, W, C như trên. Nếu ta cần sửa lỗi, thì với k , chúng ta có thể xây dựng một hàm giải mã D để **sửa tất cả các lỗi** truyền có trọng lượng $\leq k$, khi và chỉ khi khoảng cách tối thiểu giữa các từ của tập mã không nhỏ hơn $2k + 1$.

Thí dụ. Với mã chập 3, $W = \{00, 01, 10, 11\}$, khi đó $C = \{000000, 010101, 101010, 111111\}$, khoảng cách tối thiểu các từ thuộc C là 3. Cho $w = 00$, khi đó $S(000000, 1) = \{000000, 100000, 010000, 001000, 000100, 000010, 000001\}$ đều được giải mã thành 00; $w = 10$, khi đó $S(101010, 1) = \{101010, 001010, 100010, 101000, 111010, 101110, 101011\}$ đều được giải mã thành 10 (cả 2 trường hợp có $k = 1$, nên $k + 1 = 2 < 3$; $2k + 1 = 3$, nên phát hiện và sửa được lỗi). Tiếp hai hình cầu $S(010101, 1)$, $S(111111, 1)$ trong $\mathbb{Z}_2^6$ thì 4 hình cầu mới có 28 phần tử, vẫn còn 36 phần tử không thuộc hình cầu nào. Tuy nhiên nếu $w = 10$, mã hóa thành 101010 và truyền đi, nhưng nhận được là 100000, khoảng cách $d(101010, 100000) = k = 2$, $k + 1 = 3$, $2k + 1 = 5$ vậy ta phát hiện được lỗi, nhưng không sửa được (giải mã 100000 thành 00 thì không đúng).

3) Matrix sinh và kiểm tra tính chẵn lẻ.

Lấy matrix $G = [I \mid A]$ trên $\mathbb{Z}_2$, chẳng hạn $G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$. Ta mã hóa từ $w =$

$= w_1 w_2 w_3 \in \mathbb{Z}_2^3$ thành $E(w) = w.G = w_1 w_2 w_3 w_4 w_5 w_6 \in \mathbb{Z}_2^6$, với $w_4 = w_1 + w_3$; $w_5 = w_1 + w_2$; $w_6 = w_2 + w_3$. Ta có $E(\{000, 001, 010, 011, 100, 101, 110, 111\}) = \{000000, 001101, 010011, 011110, 100110, 101011, 110101, 111000\}$, khi giải mã chỉ cần xóa đi 3 bit cuối của từ mã. Khoảng cách tối thiểu giữa các từ mã là 3, nên ta có thể phát hiện lỗi có trọng lượng ≤ 2 và sửa lỗi những từ mã có 1 bit bị lỗi.

Ta tìm được $H = [B \mid I] = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$ mà $H.E(w)^T = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. Rõ ràng là $B = A^T$

và $r \in \mathbb{Z}_2^6$ là mã của từ $w \in \mathbb{Z}_2^3$ nào đó khi và chỉ khi $H.r^T = [0 \ 0 \ 0]^T$. Với từ mã là c , tính $H.c^T$ là phát hiện ra lỗi.

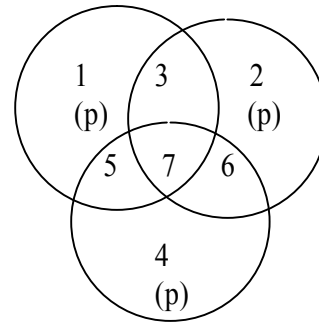
Mã Hamming

Trong viễn thông (telecommunication), **mã Hamming** là một mã sửa lỗi tuyến tính (linear error-correcting code), được đặt tên theo tên của người phát minh ra nó, Richard Hamming. Mã Hamming có thể phát hiện một bit hoặc hai bit bị lỗi (single and double-bit errors). Mã Hamming còn có thể sửa các lỗi do một bit bị sai gây ra. Ngược lại với mã của ông, mã chẵn lẻ (parity code) đơn giản vừa không có khả năng phát hiện các lỗi khi 2 bit cùng một lúc bị hoán vị (0 thành 1 và ngược lại), vừa không thể giúp để sửa được các lỗi mà nó phát hiện được.

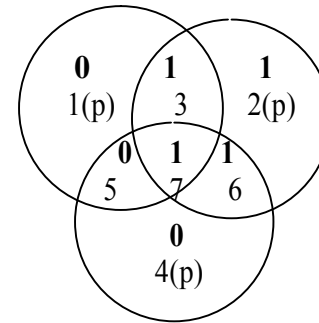
Càng nhiều bit sửa lỗi thêm vào trong thông điệp, và các bit ấy được bố trí theo một cách là mỗi bố trí của nhóm các bit bị lỗi tạo nên một hình thái lỗi riêng biệt, thì chúng ta có thể xác định được những bit bị sai. Trong một thông điệp dài 7-bit, chúng ta có 7 khả năng một bit có thể bị lỗi, như vậy, chỉ cần 3 bit kiểm tra ($2^3 = 8$) là chúng ta có thể, không những chỉ xác định được là lỗi trong truyền thông có xảy ra hay không, mà còn có thể xác định được bit nào là bit bị lỗi.

Hamming nghiên cứu các kế hoạch mã hóa hiện có, bao gồm cả mã hai-trong-năm, rồi tổng quát hóa khái niệm của chúng. Khởi đầu, ông xây dựng một [danh mục](#) (*nomenclature*) để diễn tả hệ thống máy, bao gồm cả số lượng bit dùng cho dữ liệu và các bit sửa lỗi trong một khối. Chẳng hạn, bit chẵn lẻ phải thêm 1 bit vào trong mỗi từ dữ liệu (*data word*). Hamming diễn tả phương pháp này là mã (8,7). Nó có nghĩa là một từ dữ liệu có tổng số bit là 8 bit, trong đó chỉ có 7 bit là các bit của dữ liệu mà thôi. Theo phương pháp suy nghĩ này, mã tái diễn (nhắc lại) ở trên phải được gọi là mã (3,1). Tỷ lệ thông tin là tỷ lệ được tính bằng việc lấy con số thứ hai chia cho con số thứ nhất. Như vậy với mã tái diễn (3,1) ở trên, tỷ lệ thông tin của nó là 1/3.

Hamming còn phát hiện ra nan đề với việc đảo giá trị của hai hoặc hơn hai bit nữa và



Mô hình của một mã 7-bit, bao gồm 4 bit dữ liệu (3,5,6,7) và 3 bit chẵn lẻ (1,2,4). Sự liên quan của các bit dữ liệu với bit chẵn lẻ được biểu hiện bằng các phần của hình tròn gối lên nhau. Bit thứ 1 kiểm tra bit thứ (3, 5, 7), trong khi bit 2 kiểm tra bit (3, 6, 7). Lưu ý, các vị trí (1,2,4 v.v.) thực ra là vị trí 2^0 , 2^1 , 2^2 v.v.



Các bit dữ liệu và bit chẵn lẻ trong mối quan hệ chồng gối với nhau. Các bit chẵn lẻ được tính dùng quy luật "số chẵn". Giá trị của nhóm dữ liệu là 1100110 - các bit chẵn lẻ được in đậm, và đọc từ phải sang trái.

miêu tả nó là "khoảng cách" (*distance*) (hiện nay nó được gọi là *khoảng cách Hamming* (*Hamming distance*) - theo cái tên của ông). Mã chẵn lẻ có khoảng cách bằng 2, vì nếu có 2 bit bị đảo ngược thì lỗi trong truyền thông trở nên vô hình, không phát hiện được. Mã tái diễn (3,1) có khoảng cách là 3, vì 3 bit, trong cùng một bộ ba, phải bị đổi ngược trước khi chúng ta được một từ mã khác. Mã tái diễn (4,1) (mỗi bit được nhắc lại 4 lần) có khoảng cách bằng 4, nên nếu 2 bit trong cùng một nhóm bị đảo ngược thì lỗi đảo ngược này sẽ đi thoát mà không bị phát hiện.

Cùng một lúc, Hamming quan tâm đến hai vấn đề; tăng khoảng cách và đồng thời tăng tỷ lệ thông tin lên, càng nhiều càng tốt. Trong những năm thuộc niên kỷ 1940, ông đã xây dựng một số kế hoạch mã hóa. Những kế hoạch này đều dựa trên những mã đang tồn tại song được nâng cấp và tiến bộ một cách sâu sắc. Bí quyết chia khóa cho tất cả các hệ thống của ông là việc cho các bit chẵn lẻ gối lên nhau (*overlap*), sao cho chúng có khả năng tự kiểm tra lẫn nhau trong khi cùng kiểm tra được dữ liệu nữa.

Thuật toán cho việc sử dụng bit chẵn lẻ trong 'mã Hamming' *thông thường* cũng tương đối đơn giản:

1. Tất cả các bit ở vị trí là các số mũ của 2 (*powers of two*) được dùng làm bit chẵn lẻ. (các vị trí như 1, 2, 4, 8, 16, 32, 64 v.v. hay nói cách khác $2^0, 2^1, 2^2, 2^3, 2^4, 2^5, 2^6$ v.v.)
2. Tất cả các vị trí bit khác được dùng cho dữ liệu sẽ được mã hóa. (các vị trí 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, etc.)
3. Mỗi bit chẵn lẻ tính giá trị chẵn lẻ cho một số bit trong từ mã (*code word*). Vị trí của bit chẵn lẻ quyết định chuỗi các bit mà nó luân phiên kiểm tra và bỏ qua (*skips*).
 - Vị trí 1 ($n=1$): bỏ qua 0 bit($n-1$), kiểm 1 bit(n), bỏ qua 1 bit(n), kiểm 1 bit(n), bỏ qua 1 bit(n), v.v.
 - Vị trí 2 ($n=2$): bỏ qua 1 bit($n-1$), kiểm 2 bit(n), bỏ qua 2 bit(n), kiểm 2 bit(n), bỏ qua 2 bit(n), v.v.
 - Vị trí 4 ($n=4$): bỏ qua 3 bit($n-1$), kiểm 4 bit(n), bỏ qua 4 bit(n), kiểm 4 bit(n), bỏ qua 4 bit(n), v.v.
 - Vị trí 8 ($n=8$): bỏ qua 7 bit($n-1$), kiểm 8 bit(n), bỏ qua 8 bit(n), kiểm 8 bit(n), bỏ qua 8 bit(n), v.v.
 - Vị trí 16 ($n=16$): bỏ qua 15 bit($n-1$), kiểm 16 bit(n), bỏ qua 16 bit(n), kiểm 16 bit(n), bỏ qua 16 bit(n), v.v.
 - Vị trí 32 ($n=32$): bỏ qua 31 bit($n-1$), kiểm 32 bit(n), bỏ qua 32 bit(n), kiểm 32 bit(n), bỏ qua 32 bit(n), v.v.
 - và tiếp tục như trên.

Nói cách khác, bit chẵn lẻ tại vị trí 2^k kiểm các bit ở các bit ở vị trí t có giá trị logic của phép toán AND giữa k và t là khác 0

Ví dụ dùng (11,7) mã Hamming

Lấy ví dụ chúng ta có một từ dữ liệu dài 7 bit với giá trị là "0110101". Để chứng minh phương pháp các mã Hamming được tính toán và được sử dụng để kiểm tra lỗi, xin xem bảng liệt kê dưới đây. Chữ **d** (*data*) được dùng để biểu thị các bit dữ liệu và chữ **p** (*parity*) để biểu thị các [bit chẵn lẻ](#) (*parity bits*).

Đầu tiên, các bit của dữ liệu được đặt vào vị trí tương thích của chúng, sau đó các bit chẵn lẻ cho mỗi trường hợp được tính toán dùng quy luật bit chẵn lẻ [số chẵn](#)^[1].

Thứ tự bit	1	2	3	4	5	6	7	8	9	10	11
Vị trí bit chẵn lẻ và các bit dữ liệu	p ₁	p ₂	d ₁	p ₄	d ₂	d ₃	d ₄	p ₈	d ₅	d ₆	d ₇
Nhóm dữ liệu (không có bit chẵn lẻ):			0		1	1	0		1	0	1
p ₁	1		0		1		0		1		1
p ₂		0	0			1	0			0	1
p ₄				0	1	1	0				
p ₈								0	1	0	1
Nhóm dữ liệu (với bit chẵn lẻ):	1	0	0	0	1	1	0	0	1	0	1

Cách tính các bit chẵn lẻ trong mã Hamming (từ trái sang phải). Nhóm dữ liệu mới (*new data word*) - bao gồm các bit chẵn lẻ - bây giờ là "10001100101". Nếu chúng ta thử cho rằng bit cuối cùng bị thoái hóa (*gets corrupted*) và bị lộn ngược từ 1 sang 0. Nhóm dữ liệu

mới sẽ là "10001100100"; Dưới đây, chúng ta sẽ phân tích quy luật kiến tạo mã Hamming bằng cách cho bit chẵn lẻ giá trị 1 khi kết quả kiểm tra dùng quy luật **số chẵn** bị sai.

Thứ tự bit	1	2	3	4	5	6	7	8	9	10	11		
Vị trí bit chẵn lẻ và các bit dữ liệu	p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4	d_5	d_6	d_7	Kiểm tra chẵn lẻ	Bit chẵn lẻ
Nhóm dữ liệu nhận được	1	0	0	0	1	1	0	0	1	0	0	1	
p_1	1		0		1		0		1		0	Sai	1
p_2		0	0			1	0			0	0	Sai	1
p_4				0	1	1	0					Đúng	0
p_8								0	1	0	0	Sai	1

Kiểm tra các bit chẵn lẻ (bit bị đảo lộn có nền thẫm)

chỉ số p_k với $k=2$ mũ $0,1,2,3\dots m$ (p_1 là lấy 1 bỏ 1, p_2 là lấy 2 bỏ 2, p_4 lấy 4 bỏ 4)

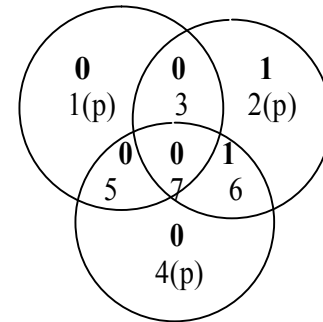
Bước cuối cùng là định giá trị của các bit chẵn lẻ (nên nhớ bit nằm dưới cùng được viết về bên phải - viết ngược lại từ dưới lên trên). Giá trị số nguyên của các bit chẵn lẻ là $11_{(10)}$, và như vậy có nghĩa là bit thứ 11 trong nhóm dữ liệu (*data word*) - bao gồm cả các bit chẵn lẻ - là bit có giá trị không đúng, và bit này cần phải đảo ngược lại.

	p_8	p_4	p_2	p_1	
Nhi phân	1	0	1	1	
Thập phân	8		2	1	$\Sigma = 11$

Việc đảo ngược giá trị của bit thứ 11 làm cho nhóm 10001100100 trở lại thành 10001100101.

Bằng việc bỏ đi phần mã Hamming, chúng ta lấy được phần dữ liệu gốc với giá trị là 0110101.

Lưu ý, các bit chẵn lẻ không kiểm tra được lẫn nhau, nếu chỉ một bit chẵn lẻ bị sai thôi, trong khi tất cả các bit khác là đúng, thì chỉ có bit chẵn lẻ nói đến là sai mà thôi và không phải là các bit nó kiểm tra (*not any bit it checks*).



Khi hai bit dữ liệu (3,7) có cùng bit chẵn lẻ kiểm tra tại vị trí 2^k - ví dụ (1,2) - biến đổi giá trị (lỗi trong truyền thông) thì giá trị của bit chẵn lẻ vẫn đúng như giá trị gốc (0,1)

Cuối cùng, giả sử có hai bit biến đổi, tại vị trí x và y . Nếu x và y có cùng một bit tại vị trí 2^k trong đại diện nhị phân của chúng, thì bit chẵn lẻ tương ứng với vị trí đây kiểm tra cả hai bit, và do đó sẽ giữ nguyên giá trị, không thay đổi. Song *một số* bit chẵn lẻ nào đây nhất định phải bị thay đổi, vì $x \neq y$, và do đó hai bit tương ứng nào đó có giá trị x và y khác nhau. Do vậy, mã Hamming phát hiện tất cả các lỗi do hai bit bị thay đổi — song nó không phân biệt được chúng với các lỗi do 1 bit bị thay đổi.

Mã Hamming (7,4)

Hiện thời, khi nói đến **mã Hamming** chúng ta thực ra là muốn nói đến mã (7,4) mà Hamming công bố năm 1950. Với mỗi nhóm 4 bit dữ liệu, mã Hamming thêm 3 bit kiểm tra. Thuật toán (7,4) của Hamming có thể sửa chữa bất cứ một bit lỗi nào, và phát hiện tất cả lỗi của 1 bit, và các lỗi của 2 bit gây ra. Điều này có nghĩa là đối với tất cả các phương tiện truyền thông không có chùm lỗi đột phát (*burst errors*) xảy ra, mã (7,4) của Hamming rất có hiệu quả (trừ phi phương tiện truyền thông có độ nhiễu rất cao thì nó mới có thể gây cho 2 bit trong số 7 bit truyền bị đảo lộn).

Ví dụ về cách dùng các ma trận thông qua GF(2) [\[2\]](#)

Nguyên lý của mã Hamming bắt nguồn từ việc khai triển và mở rộng quan điểm chẵn lẻ. Việc khai triển này bắt đầu bằng việc nhân các ma trận, được gọi là **Ma trận Hamming** (*Hamming matrices*), với nhau. Đối với mã Hamming (7,4), chúng ta sử dụng hai ma trận có liên quan gần gũi, và đặt tên cho chúng là:

$$H_e := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix} \text{ và } H_d := \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

Các cột vectơ trong H_e là nền tảng hach của H_d và phần trên của H_e (4 hàng đầu) là một ma trận đơn vị (*identity matrix*). Ma trận đơn vị cho phép vectơ dữ liệu đi qua trong khi làm tính nhân, và như vậy, các bit dữ liệu sẽ nằm ở 4 vị trí trên cùng (sau khi nhân). Sau khi phép nhân hoàn thành, khác với cách giải thích ở phần trước (các bit chẵn lẻ nằm ở vị trí 2^k), trật tự của các bit trong từ mã (*codewords*) ở đây khác với cách bố trí đã nói (các bit dữ liệu nằm ở trên, các bit kiểm chẵn lẻ nằm ở dưới).

Chúng ta dùng một nhóm 4 bit dữ liệu (số 4 trong cái tên của mã là vì vậy) chủ chốt, và cộng thêm vào đó 3 bit dữ liệu thừa (vì $4+3=7$ nên mới có số 7 trong cái tên của mã). Để truyền gửi dữ liệu, chúng ta hãy nhóm các bit dữ liệu mà mình muốn gửi thành một vectơ. Lấy ví dụ, nếu dữ liệu là "1011" thì vectơ của nó là: $p = [1 \ 0 \ 1 \ 1]^T$. Giả sử, chúng ta muốn truyền gửi dữ liệu trên. Chúng ta tìm tích của H_e và p , với các giá trị môđulô 2 [\[3\]](#):

$$H_e.p = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = r$$

Máy thu sẽ nhân H_d với r , để kiểm tra xem có lỗi xảy ra hay không. Thực hiện tính nhân này, máy thu được các giá trị đồng dư môđulô 2:

$$H_d.r = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Vì chúng ta được một vectơ toàn số không cho nên máy thu có thể kết luận là không có lỗi xảy ra. Sở dĩ một vectơ toàn số không có nghĩa là không có lỗi, bởi vì khi H_e được nhân với vectơ dữ liệu, một sự thay đổi trong nền tảng xảy ra đối với không gian bên trong vectơ (*vector subspace*), tức là hach của H_d . Nếu không có vấn đề gì xảy ra trong khi

truyền thông, r sẽ nằm nguyên trong hạch của H_d và phép nhân sẽ cho kết quả một vector toàn số không.

Trong một trường hợp khác, nếu chúng ta giả sử là lỗi một bit đã xảy ra. Trong toán học, chúng ta có thể viết: $r + e_i$ môđulô 2, trong đó e_i là vector đơn vị đứng thứ i (i -th *unit vector*), có nghĩa là, một vector số 0 có một giá trị một trong vị trí i (tính từ 1 tính đi). Biểu thức trên nói cho chúng ta biết rằng có một bit bị lỗi tại vị trí i .

Nếu bây giờ chúng ta nhân H_d với cả hai vector này:

$$H_d(r + e_i) = H_d r + H_d e_i$$

Vì r là dữ liệu thu nhận được không có lỗi, cho nên tích của H_d và r bằng 0. Do đó

$$H_d r + H_d e_i = 0 + H_d e_i = H_d e_i$$

Vậy, tích của H_d với vector nền chuẩn tại cột thứ i (*the i th standard basis vector*) làm lộ ra cột ở trong H_d , vì thế mà chúng ta biết rằng lỗi đã xảy ra tại vị trí cột này trong H_d . Vì chúng ta đã kiến tạo H_d dưới một hình thức nhất định, cho nên chúng ta có thể hiểu giá trị của cột này như một số nhị phân - ví dụ, (1,0,1) là một cột trong H_d , tương đồng giá trị với cột thứ 5, do đó chúng ta biết lỗi xảy ra ở đâu và có thể sửa được nó.

Lấy ví dụ, giả sử chúng ta có:

$$s = r + e_2 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}. \text{ Nếu thực hiện phép nhân: } H_d s = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

Tích của phép nhân cho chúng ta một kết quả tương đương với cột thứ 2 ("010" tương đương với giá trị 2 trong số thập phân), và do đó, chúng ta biết rằng lỗi đã xảy ra ở vị trí thứ 2 trong hàng dữ liệu, và vì vậy có thể sửa được lỗi.

Chúng ta có thể dễ dàng thấy rằng, việc sửa lỗi do 1 bit bị đảo lộn gây ra, dùng phương pháp trên là một việc thực hiện được. Bên cạnh đó, mã Hamming còn có thể phát hiện lỗi do 1 bit hoặc 2 bit bị đảo lộn gây ra, dùng tích của H_d khi tích này không cho một vector số không. Tuy thế, song mã Hamming không thể hoàn thành cả hai việc.

Mã Hamming và bit chẵn lẻ bổ sung

Nếu chúng ta bổ sung thêm một bit vào mã Hamming, thì mã này có thể dùng để phát hiện những lỗi gây ra do 2 bit bị lỗi, và đồng thời nó không cản trở việc sửa các lỗi do một bit gây ra. [1] Nếu không bổ sung một bit vào thêm, thì mã này có thể phát hiện các lỗi do một bit, hai bit, ba bit gây ra, song nó sẽ cản trở việc sửa các lỗi do một bit bị đảo lộn. Bit bổ sung là bit được áp dụng cho tất cả

7 bit dữ liệu	byte có bit chẵn lẻ	
	Quy luật số chẵn	Quy luật số lẻ
0000000	00000000	00000001
1010001	10100011	10100010
1101001	11010010	11010011
1111111	11111111	11111110

các bit sau khi tất cả các bit kiểm của mã Hamming đã được thêm vào.

Khi sử dụng tính sửa lỗi của mã, nếu lỗi ở một bit chẵn lẻ bị phát hiện và mã Hamming báo hiệu là có lỗi xảy ra thì chúng ta có thể sửa lỗi này, song nếu chúng ta không phát hiện được lỗi trong bit chẵn lẻ, nhưng mã Hamming báo hiệu là có lỗi xảy ra, thì chúng ta có thể cho

rằng lỗi này là do 2 bit bị đổi cùng một lúc. Tuy chúng ta phát hiện được nó, nhưng không thể sửa lỗi được.

- **Mã Rabin.** Lấy 2 số nguyên tố p, q khá lớn (hàng trăm chữ số thập phân), người gửi tin biết số $n = pq$ và mã tập tin M theo hàm $E(M) \equiv M^2 \pmod{n}$, người nhận tin biết cặp số (p, q) và giải phương trình $M^2 \equiv E(M) \pmod{n}$ tìm M . Người thứ ba biết n và $E(M)$ thì cũng rất khó tìm ra cặp số (p, q) để giải mã.

Thí dụ. Chọn $p = 10007$; $q = 10657$; $n = p \cdot q = 106644599$; $M = \text{Sum}[A[i]] \cdot 100^{\text{Length}[A]-i}, \{i, 1, \text{Length}[A]\} = 15001700011411140803$; $E(M) = \text{Mod}[M^2, n] = 77921277$.

5) Mã khóa công khai (Mã RSA).

- Một hệ thống mã khóa công khai khác được gọi là hệ thống mã *RSA*, được đặt theo tên của những người phát minh ra nó: *R. Rivest, A. Shamir và L. Adleman*. Phương pháp này dựa trên Định lý Euler, mở rộng Định lý nhỏ của Fermat.

*) Mã hóa và giải mã *RSA*: Lấy hai số nguyên tố p và q đủ lớn (chẳng hạn có 500 chữ số thập phân) và tính $n = pq$, $\varphi(n) = n(1 - 1/p)(1 - 1/q) = (p - 1)(q - 1)$. Lấy ngẫu nhiên một số tự nhiên khá lớn là e (nhỏ hơn n và nguyên tố cùng nhau với n , tức là $e \in \mathbb{Z}_{\varphi(n)}^*$) và tính $d = e^{-1}$ trong $\mathbb{Z}_{\varphi(n)}^*$ (d có tính chất $ed \equiv 1 \pmod{\varphi(n)}$).

Mã hóa $N = E(M) = M^e \pmod{n}$ và giải mã $D(N) = N^d \pmod{n} = M^{ed} = M^{k\varphi(n)+1} = M \pmod{n}$.

Thí dụ. Chọn các số nguyên tố $p = 1231$ và $q = 337$ và tính $n = pq = 414847$. Tính $\varphi(n) = (p - 1)(q - 1) = 1230 \cdot 336 = 413280$; chọn e một cách ngẫu nhiên trong $\mathbb{Z}_{413280}^*$ – giả sử chọn $e = 331619$. Cuối cùng, tính toán (trong $\mathbb{Z}_{413280}^*$) tìm $d = e^{-1} = 83419$. Giả sử $M = 3456789$, khi đó $N = M^e = 244460$, $M = N^d = 138013 \pmod{n}$. Tính trong *Math*..

```
Print["{p,q}=", {p,q}={Prime[1032],Prime[458]}] (*Lấy p,q là số nguyên tố thứ
1032 và thứ 458*)
Print["n=", n=p*q]
Print["phi=", phi=(p-1)(q-1)] (*Hàm phi Euler của n*)
For[e=phi;i=0,GCD[e,phi]!=1,i++,e=Random[Integer,{n/2,n-1}]];Print["e=",e]
For[d=0;i=0,Mod[e*d,phi]!=1,i++,d+=1];Print["d=",d]
Print["M=", M=3456789]
Print["N=", n1=Mod[M^e,n]]
Mod[n1^d,n]

{p,q}={8231,3251}
n=26758981
phi=26747500
e=19431531
d=14547371
M=3456789
N=13088420
3456789
```